Python Engineering - Week 04

Week 04 - OOP, files and testing

```
01_python_bootcamp/week_04_oop_files_testing/
```

How to use this document

This PDF is a review index for the week. Each class summarizes **the problem, concepts and syntax/functions introduced**. If a concept becomes fuzzy, locate it here and return to the original exercise.

class_13_classes_objects

EX21 - VerticalState class

Object-oriented programming: class = general definition; object/instance = one concrete realization.

Attributes: Persistent object data: self.h, self.v, self.t, self.g.

Methods: Functions defined inside a class that operate on a particular object.

__init__: Initializes attributes when an instance is created.

self: Reference to the current object; earth_state.specific_energy() supplies earth_state as self.

State: euler_step(dt) mutates object attributes; different objects keep independent state.

Architecture: vertical_state.py defines the class; main.py creates earth_state/moon_state and uses methods.

class_14_file_io

EX22 - Persistence and CSV

File I/O: open(), modes r / w / a and the with open(...) context manager.

Text: write(), readline(), strip(), split() and float() conversion.

Paths: pathlib.Path, __file__, resolve(), parent and mkdir(exist_ok=True).

CSV: Serialize numbers to text and parse text back into numbers.

Validation: Round trip: data -> file -> recovered data; shape + np.allclose.

Architecture: vertical_state.py = model; file_io.py = persistence; main.py = orchestration.

Growing data: list + append for unknown N; preallocation for known N; np.vstack for joining arrays.

class_15_exceptions_debugging

EX23 - Exceptions and debugging

Goal: Respond to predictable failures without hiding them.

Exceptions: try, except, else, finally and raise.

Error types: FileNotFoundError and ValueError.

Debugging: breakpoints, Variables, Call Stack, Step Into / Step Over / Step Out and traceback.

Design: Detect near the problem; handle where enough context exists to decide.

Application: Safe loading of valid, missing or corrupted CSV data.

class_16_testing_project_structure

Testing and project structure

Goal: Close the bootcamp with automated verification and maintainable structure.

Testing: Check expected behavior through reproducible cases and assertions/tests.

Structure: Separate code, data, tests, configuration and documentation consistently.

Quality: Prevent regressions and prepare projects for portfolio/GitHub.

Outcome: Prepare the codebase for larger projects and the transition to numerical methods.

Quick concept index

Concept	Where to find it
Object-oriented programming: class = general definition; object/instance = one concrete realization.	class_13_classes_objects
Attributes: Persistent object data: self.h, self.v, self.t, self.g.	class_13_classes_objects
Methods: Functions defined inside a class that operate on a particular object.	class_13_classes_objects
__init__: Initializes attributes when an instance is created.	class_13_classes_objects
self: Reference to the current object; earth_state.specific_energy() supplies earth_state as self.	class_13_classes_objects
State: euler_step(dt) mutates object attributes; different objects keep independent state.	class_13_classes_objects
Architecture: vertical_state.py defines the class; main.py creates earth_state/moon_state and uses methods.	class_13_classes_objects
File I/O: open(), modes r / w / a and the with open(...) context manager.	class_14_file_io
Text: write(), readline(), strip(), split() and float() conversion.	class_14_file_io
Paths: pathlib.Path, __file__, resolve(), parent and mkdir(exist_ok=True).	class_14_file_io
CSV: Serialize numbers to text and parse text back into numbers.	class_14_file_io
Validation: Round trip: data -> file -> recovered data; shape + np.allclose.	class_14_file_io
Architecture: vertical_state.py = model; file_io.py = persistence; main.py = orchestration.	class_14_file_io
Growing data: list + append for unknown N; preallocation for known N; np.vstack for joining arrays.	class_14_file_io
Goal: Respond to predictable failures without hiding them.	class_15_exceptions_debugging
Exceptions: try, except, else, finally and raise.	class_15_exceptions_debugging
Error types: FileNotFoundError and ValueError.	class_15_exceptions_debugging
Debugging: breakpoints, Variables, Call Stack, Step Into / Step Over / Step Out and traceback.	class_15_exceptions_debugging
Design: Detect near the problem; handle where enough context exists to decide.	class_15_exceptions_debugging

Review rule

First locate **the concept**; then open that class PDF; finally run or modify the exercise. Document + code is the review mechanism for this course.